SDN-Integrated Multi-Client TCP Chat Application

D1 Design Document

GitHub: https://github.com/Abhivbk/CHAT_CLIENT

Name	SRN	Contribution
Abhinav Kulkarni	PES1UG25CS014	Chat application: server, client, protocol, telemetry, tests
Akshith Singh	PES1UG25CS048	SDN: Ryu controller, policies, Baseline vs SDN comparison
Amogh B	PES1UG25CS053	Mininet: topology, QoS queues, experiment scripts

1. What we have done

We built a multi-client chat application using raw TCP sockets and ran it inside a Mininet network controlled by a Ryu (OpenFlow 1.3) SDN controller. The chat server sends a small UDP message to the controller whenever something happens (a user registers, joins a room, sends a message and so on). The controller uses these messages to give some chat connections better treatment on a congested link. We then compared the network with and without the controller policies (Baseline and SDN).

2. How it is built

- **Chat server (server.py).** Python standard library only. It runs one thread per client on TCP port 5000 and uses locks to protect the user list, room list and socket writes. Messages are plain text, one command per line, with a maximum of 1024 characters. Commands: /register, /users, /msg, /create, /join, /leave, /rooms, /roommsg, /help, /quit. A room must be created before it can be joined, and empty rooms are deleted.
- **Telemetry (event_emitter.py).** For every event the server sends one UDP JSON packet to the controller (connect, register, private_msg, room_join, room_leave, room_msg, disconnect). It contains the client's IP and TCP port, for example
  `{"event":"room_join","user":"alice","room":"vip","ip":"10.0.0.2","port":48122}`
- **Client and tests (client.py, test_clients.py).** The client works interactively or from a script file. The test program has three modes: functional (pass/fail checks), rtt (private message latency) and scale (many clients broadcasting in one room). Results can be saved as CSV.
- **Topology (sdn/topology.py).** h1 (server, 10.0.0.1) connects to switch s1, which connects to s2 over a 10 Mbit/s link. Clients h2 and h3 and the traffic generator h4 connect to s2. Open vSwitch queues on the bottleneck link: queue 0 is the default, queue 1 is for priority traffic (60% minimum) and queue 2 is for throttled traffic (10% maximum).
- **Controller (sdn/sdn_controller.py).** It works as a learning switch and also listens for the UDP events on port 9999. Policy A: a connection that joins room vip or load, or registers as rttA or rttB, gets flow rules in both directions that send its packets to queue 1. Policy B: a user sending more than 40 messages per second is moved to queue 2 for 10 seconds. Rules are removed when the user leaves or disconnects.

3. Design decisions

- UDP telemetry is sent separately from the chat connection, because a switch cannot read chat commands and UDP will not slow the server down if the controller is unavailable. The drawback is that events can be lost.
- Priority is given per TCP connection (source IP and port), since all messages of a user travel on one connection and cannot be separated. This is why the server reports the client's IP and port.
- One thread per client with blocking sockets keeps the code simple and is enough for the number of clients we tested.
- Baseline and SDN runs use the same code and only differ in the controller mode, so the comparison is fair.
- Queues are set only on the bottleneck link, because that is where congestion happens.

4. Experiment and results

The 10 Mbit/s link was loaded with 12 Mbit/s of UDP traffic (iperf) in each direction. From h2 we ran the rtt test (200 messages) and the scale test (10 clients, 20 messages each), 3 times for each configuration. Average of the 3 runs:

Metric	Baseline	SDN
Delivery latency, average (ms)	1.95	3.46
Delivery latency, 95th percentile (ms)	5.65	12.22
Delivery latency, maximum (ms)	26.1	29.3
Messages lost (scale test)	0	0

No messages were lost in either case. In two of the three SDN runs the 95th percentile latency (2.9 ms and 3.3 ms) was lower than in all baseline runs (4.2 to 7.4 ms), but the third SDN run was much higher (30.5 ms), which makes the SDN average worse. With only 3 runs we cannot say that SDN gives a reliable improvement. The switch statistics show that the chat traffic used queue 1 and the iperf traffic used queue 0. Raw data and the graph are in sdn/results/.

5. How to build and run

The chat application needs only Python 3.8 or newer. The SDN part needs Ubuntu 22.04 with Mininet, Open vSwitch, iperf and Ryu (installed by the script below).

A. Chat only (any machine):

```
git clone https://github.com/Abhivbk/CHAT_CLIENT
cd CHAT_CLIENT
python3 server.py                                    # terminal 1
python3 client.py --host 127.0.0.1 --user alice      # terminal 2
python3 client.py --host 127.0.0.1 --user bob        # terminal 3
python3 test_clients.py functional --host 127.0.0.1  # automated checks
```

Commands to try in a client: /create vip, /join vip, /roommsg vip hello, /msg bob hi, /help

B. Full SDN experiment (Ubuntu VM):

```
cd CHAT_CLIENT/sdn
bash scripts/install_deps.sh                 # one time only
bash scripts/cleanup.sh
# Terminal 1 (controller):
bash scripts/start_controller.sh sdn         # use 'baseline' for the baseline run
# Terminal 2 (network and experiment, takes 5-10 minutes):
sudo python3 topology.py --experiment sdn --chat-dir ~/CHAT_CLIENT
# Run both terminals again with 'baseline', then:
python3 compare_results.py                   # creates results/comparison.png
```

For a manual demo, run `sudo python3 topology.py` without --experiment and open `xterm h1 h2 h3`. Start the server on h1 with `python3 server.py --controller 10.0.0.100:9999` and the clients on h2 and h3 with `--host 10.0.0.1`. When a client joins room vip, the controller prints "POLICY A" and `bash scripts/show_flows.sh` shows the set_queue rules. Run `bash scripts/cleanup.sh` if anything is left over from a previous run.

6. Limitations

Priority works per connection and not per message. The telemetry is UDP without authentication. The results are from only 3 runs on a virtual network, so they are only indicative.